INFORMATIONSBLATT

IB01 · Bash-Skripte – Grundlagen

🎯 Lernziel

Nach diesem Informationsblatt können Sie:

- den Aufbau eines Bash-Skripts erklären (Shebang, Kommentare)
- Variablen definieren und ihren Wert ausgeben
- Befehlssubstitution (`$(befehl)`) anwenden
- Skripte ausführbar machen (`chmod +x`) und starten
- ein vollständiges erstes Mini-Skript eigenständig schreiben

1 · Was ist ein Bash-Skript?

Ein Bash-Skript ist eine Textdatei, die eine Abfolge von Shell-Befehlen enthält. Diese Befehle werden der Reihe nach ausgeführt, als würden Sie sie einzeln im Terminal eingeben – nur vollautomatisch und wiederholbar.

Skripte eignen sich ideal für Aufgaben, die regelmäßig und immer gleich ausgeführt werden sollen: Backups anlegen, Dateien bereinigen, Dienste überwachen.

1.1 | Die Shebang-Zeile

Jedes Bash-Skript sollte mit der sogenannten **Shebang-Zeile** beginnen. Sie teilt dem Betriebssystem mit, welches Programm das Skript ausführen soll.

```
Shebang-Zeile – immer die erste Zeile im Skript

#!/bin/bash
```

💡 Merke

Mit der Shebang-Zeile ist eindeutig festgelegt, dass `/bin/bash` das Skript ausführt, sobald es direkt als Programm gestartet wird (`./skript.sh` , `/opt/scripts/skript.sh` , per Cron). Fehlt sie, entscheidet die aufrufende Umgebung über den Interpreter (z. B. `sh` statt `bash`) – Bash-typische Befehle können dann unerwartet fehlschlagen. Mit `bash skript.sh` wird Bash dagegen immer ausdrücklich gestartet.

1.2 | Kommentare

Alles hinter einem `#` -Zeichen (außer der Shebang-Zeile) ist ein Kommentar und wird nicht ausgeführt. Kommentieren Sie Ihre Skripte immer – auch für Ihr späteres Ich.

Aufbau eines kommentierten Skripts

```
#!/bin/bash
# ============================================================
# skript_name.sh – Kurzbeschreibung
# Autor:   Max Mustermann
# Datum:   2025-03-15
# Zweck:   Was macht dieses Skript?
# ============================================================

echo "Skript gestartet."   # Inline-Kommentar: wird auch ignoriert
```

2 · Variablen

2.1 | Variablen definieren und verwenden

Variablen speichern Werte, die im Skript mehrfach verwendet werden. Wichtig: **kein Leerzeichen** rund um das `=`-Zeichen.

Aktion	Syntax	Beispiel
Variable definieren	`NAME=Wert`	`ZIEL="/backup"`
Variable lesen	`$NAME` oder `${NAME}`	`echo "$ZIEL"`
Befehlsausgabe speichern	`VAR=$(befehl)`	`DATUM=$(date +%Y-%m-%d)`

Variablen – Beispiel aus dem Backup-Kontext

```
#!/bin/bash
QUELLE="/var/www/html/netconsult"
ZIEL="/backup/web"
AUFBEWAHRUNG=7
DATUM=$(date +%Y-%m-%d)        # z. B. "2025-03-15"
ARCHIV="$ZIEL/web_$DATUM.tar.gz"

echo "Backup wird erstellt: $ARCHIV"
```

⚠️ Häufiger Fehler: Leerzeichen bei Zuweisung

```
ZIEL = "/backup"    # FALSCH – Bash interpretiert dies als Befehlsaufruf
ZIEL="/backup"      # RICHTIG
```

2.2 | Wichtige vordefinierte Variablen

Variable	Bedeutung	Beispielwert
`$0`	Name des Skripts	`/opt/scripts/backup.sh`
`$1`, `$2`, …	Übergebene Parameter	`./backup.sh /var/www` → `$1` = `/var/www`
`$?`	Exit-Code des letzten Befehls	`0` = Erfolg, `≠ 0` = Fehler
`$$`	Prozess-ID des aktuellen Skripts	`12345`
`$HOME`	Home-Verzeichnis des Benutzers	`/home/admin`

3 · Skript ausführbar machen und starten

Eine neu erstellte Skript-Datei hat zunächst keine Ausführungsberechtigung. Diese muss explizit vergeben werden:

Skript ausführbar machen

```
# Ausführungsrecht vergeben (für Eigentümer, Gruppe und Andere)
chmod +x /opt/scripts/backup.sh

# Alternativ: explizite Rechtevergabe (Eigentümer: rwx, Gruppe: r-x, Andere: ---)
chmod 750 /opt/scripts/backup.sh

# Skript starten
./backup.sh              # aus dem gleichen Verzeichnis
/opt/scripts/backup.sh   # mit absolutem Pfad (empfohlen für Cronjobs)
```

Befehl	Bedeutung
`chmod +x datei`	Fügt Ausführungsrecht für alle hinzu
`chmod 750 datei`	Eigentümer: rwx (7), Gruppe: r-x (5), Andere: --- (0)
`ls -l datei`	Zeigt aktuelle Berechtigungen an (`-rwxr-x---`)

4 · Ein erstes vollständiges Mini-Skript

Kombinieren Sie Shebang, Kommentare, Variablen und `echo`-Ausgaben zu einem vollständigen ersten Skript. Dieses Muster ist die Basis für alle weiteren Skripte in LS03.

hallo_server.sh – Ihr erstes Bash-Skript

```
#!/bin/bash
# ============================================================
# hallo_server.sh – Erstes Bash-Skript
# Autor:   [Ihr Name]
# Datum:   2025-09-01
# Zweck:   Systeminfo ausgeben und Skript-Grundstruktur zeigen
# ============================================================

SERVERNAME="netconsult-srv"
DATUM=$(date +%Y-%m-%d)
UHRZEIT=$(date +%H:%M:%S)

echo "============================================"
echo "Server:   $SERVERNAME"
echo "Datum:    $DATUM"
echo "Uhrzeit:  $UHRZEIT"
echo "Benutzer: $USER"
echo "============================================"
echo "Skript beendet."
```

Erstellen Sie die Datei, machen Sie sie ausführbar und starten Sie sie:

> 📁 **Zuerst: das Skript-Verzeichnis anlegen**
> Das Verzeichnis `/opt/scripts/` existiert standardmäßig **nicht** und liegt unter `/opt`, das **root** gehört. Sie müssen es deshalb **einmalig mit** `sudo` **anlegen**. Damit Sie Ihre Skripte danach *ohne* `sudo` bearbeiten können, machen Sie sich gleich selbst zum Eigentümer.

Skript-Verzeichnis einmalig anlegen

```
# Verzeichnis anlegen (gehoert zunaechst root)
sudo mkdir -p /opt/scripts

# sich selbst zum Eigentuemer machen -> danach ohne sudo bearbeitbar
sudo chown $USER:$USER /opt/scripts

# Kontrolle
ls -ld /opt/scripts
```

Skript anlegen und starten

```
# Datei anlegen
nano /opt/scripts/hallo_server.sh

# Ausführungsrecht vergeben
chmod +x /opt/scripts/hallo_server.sh

# Skript starten
/opt/scripts/hallo_server.sh
```

✅ Zwischenübung – Mini-Skript schreiben

Schreiben Sie ein Bash-Skript `system_info.sh` (ca. 8–10 Zeilen), das folgendes tut:

1. Shebang-Zeile und mindestens 2 Kommentarzeilen (Autor, Zweck)
2. Eine Variable `HOSTNAME` mit dem Wert des Befehls `hostname` befüllen
3. Eine Variable `DATUM` mit dem aktuellen Datum befüllen
4. Beide Variablen mit `echo` ausgeben
5. Das Skript ausführbar machen und testen

Erwartete Ausgabe (Beispiel):

```
Hostname: netconsult-srv
Datum:    2025-09-01
```


LF10b – Serverdienste bereitstellen und Administrationsaufgaben automatisieren · LS03 – Administrationsaufgaben mit Bash automatisieren · IB01 – Bash-Skripte: Grundlagen · Schuljahr 2025/2026